```
      <button onClick={resetTimer} disabled={seconds === 0}>
        Reset
      </button>
    </div>
  );
};
```

Listing 3.12 and 3.13 show some advanced JavaScript concepts:

- *Closures*—The `useInterval` hook uses closures to encapsulate interval logic. The `savedCallback` ref ensures that the latest callback is always used.
- *Higher-order functions*—The `useInterval` hook acts as a higher-order function by abstracting reusable interval logic into a declarative API.
- *Private fields*—The `useRef` in `useInterval` acts like a private field to store mutable data (`savedCallback`) that persists across renders but does not trigger re-renders.
- *Immutability patterns*—State updates are handled immutably using functional updates (`setSeconds((prev) => prev + 1)`).
- *ES2015+ features*—Arrow functions, template literals, and restructuring.

TIP

Stale state is a common pitfall in React state management, especially during interviews. Functional updates (such as setSeconds((prev) => prev + 1)) should be used whenever the new state depends on the previous state—particularly when multiple state updates may occur rapidly or when updates are batched by React. This approach ensures the latest state is always referenced and helps avoid subtle bugs.

This example can be executed with the command: `VITE_NAME=Timer npm run dev`.

When time allows, this example can be enhanced by styling the UI, adding countdown functionality, and improving user feedback with visual indicators such as a progress bar or animation to indicate the timer is running.

3.6 The rising significance of TypeScript